

PORTFOLIO PROJECT

NYC Taxi Data Engineering on Azure

End-to-end pipeline using Azure Data Factory, ADLS Gen2, Databricks, Delta Lake and Gold analytics tables.

Based on Chapter 7 timestamps provided: 21:16:30 – 24:33:37



Project Objective

What this portfolio project is doing

Goal

Build a complete cloud data pipeline that ingests NYC Taxi trip data, cleans and transforms it, and publishes business-ready Delta tables for reporting and analysis.

- Practice real Azure data engineering workflow
- Show Bronze → Silver → Gold architecture
- Demonstrate PySpark, Delta and cloud storage skills

Raw taxi data



Cloud pipeline



Analytics data

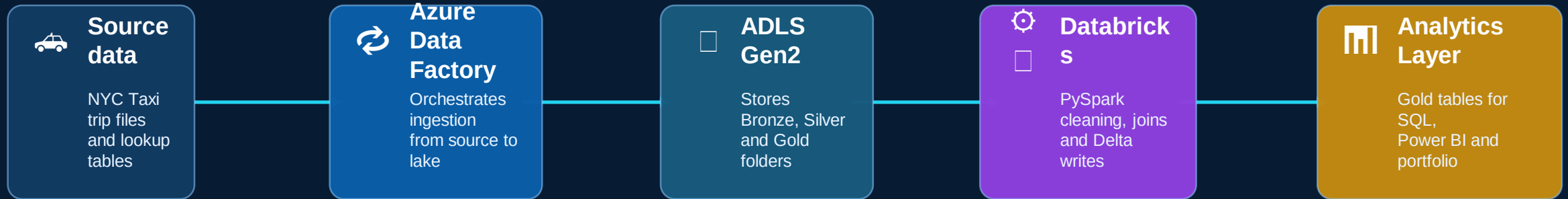


Business questions enabled

Trip patterns • payment behaviour • zone performance
• reporting-ready datasets

End-to-End Architecture

How data moves through the Azure platform



Core design pattern: Medallion Architecture

Bronze = raw data → Silver = cleaned data → Gold = business-ready tables

Data Used in the Project

NYC Taxi data is a good real-world dataset for cloud pipelines

Input files

- Trip records such as pickup/drop-off details
- Payment and fare-related fields
- Taxi zone lookup data
- Trip type / zone mapping data

Why NYC Taxi?

- Large public dataset
- Clear business context
- Good for joins and transformations
- Useful for analytics dashboards

Portfolio value

- Shows cloud storage design
- Shows PySpark skills
- Shows Delta table creation
- Shows debugging ability

Azure Services Used

Main technologies demonstrated in the project



Azure Data Factory

Pipeline orchestration and ingestion



ADLS Gen2

Data lake storage for all layers



Databricks

PySpark transformations and Delta writes



Unity Catalog

Governance, storage credentials and external locations



Delta Lake

Reliable tables with ACID-style lakehouse storage

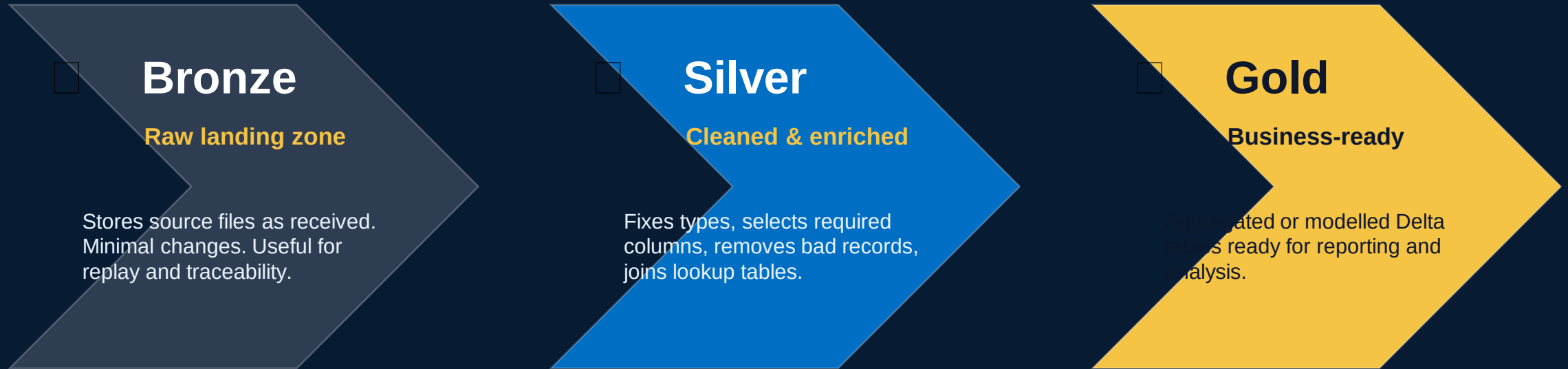


Power BI / SQL

Final consumption layer for reporting

Medallion Architecture

How the data is organised inside the data lake



This structure makes the pipeline easier to debug, rerun and explain in interviews.

Azure Data Factory Ingestion

What ADF is doing in the project

ADF pipeline responsibilities

- Connect to source data or HTTP/API file locations
- Copy NYC Taxi files into ADLS Gen2
- Parameterise paths such as year/month folders
- Run activities in sequence and monitor success/failure



Output path example

`abfss://bronze@<storage-account>.dfs.core.windows.net/...`

Databricks Transformation Layer

PySpark turns raw files into clean Delta tables

Typical PySpark steps

- Read files from ADLS using ABFSS path
- Inspect schema and fix data types
- Select useful columns for analytics
- Join trip data with zone lookup tables
- Write cleaned results to Delta tables

Example transformation logic

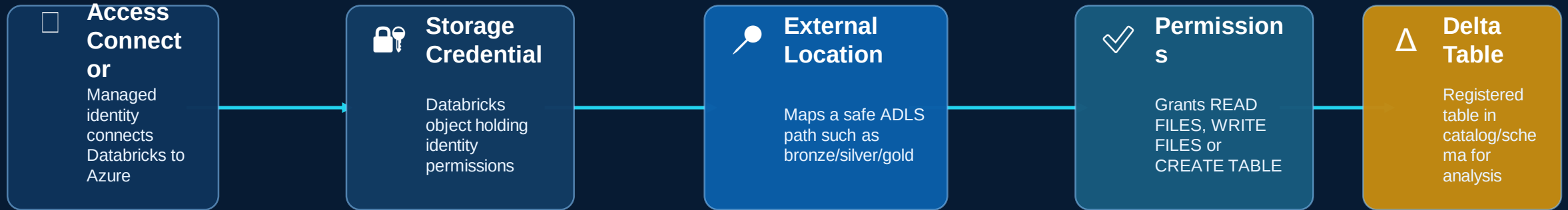
```
from pyspark.sql.functions import split, col

df_zone = df_trip_type.withColumn(
    "zone1", split(col("Zone"), "/").getItem(0)
).withColumn(
    "zone2", split(col("Zone"), "/").getItem(1)
)

df_zone.write.format("delta")\
    .mode("append")\
    .saveAsTable("gold.trip_zone")
```

Unity Catalog, Storage Credentials & External Locations

Access control required before Databricks can read/write lake data



Important learning: Storage permission and external-location path must be correct before writing Delta tables.

Gold Layer Output

Business-ready tables to show in portfolio

Gold tables created

- gold.trip_zone
- gold.fact_trips
- gold.dim_location
- gold.dim_payment_type
- gold.summary_by_zone

What Gold means

- Clean column names
- Reliable table format
- Ready for SQL queries
- Ready for Power BI model
- Reusable for dashboards

Sample insights

- Trips by borough or zone
- Average fare by payment type
- Pickup/drop-off demand areas
- Monthly trend analysis
- Data-quality checks

Problems Solved During Implementation

These issues make the project stronger for your portfolio

External-location overlap

→ Dropped/recreated location or used a non-overlapping parent path.

Missing storage permissions

→ Assigned required Azure RBAC roles to the managed identity.

Wrong ABFSS URI

→ Fixed path syntax: abfss://container@account.dfs.core.windows.net/folder.

Delta write path error

→ Created parent external location before writing to Gold path.

Array index error in PySpark

→ Used safe split/get logic when Zone has only one part.

Typo in file format

→ Corrected praquet/praquet to parquet.

Portfolio message: I can troubleshoot real cloud-data pipeline errors, not only follow a tutorial.

How to Present This in Your Portfolio

Simple case-study wording you can reuse

Project summary

Designed and implemented an end-to-end data engineering pipeline for NYC Taxi data on Azure. The pipeline ingests raw data with Azure Data Factory, stores it in ADLS Gen2 using Bronze/Silver/Gold layers, transforms it with Databricks PySpark, and publishes Delta tables for analytics and reporting.

Tools: Azure Data Factory • ADLS Gen2 • Databricks • PySpark • Delta Lake • Unity Catalog • Power BI

Skills demonstrated

- Cloud data ingestion and orchestration
- Data lake design using medallion architecture
- PySpark data cleaning and transformation
- Delta table creation and catalog registration
- Debugging permission, path and schema errors
- Explaining technical work clearly

30-Second Interview Explanation

Use this when someone asks about your project

“I built an Azure data engineering pipeline using NYC Taxi data. I used Azure Data Factory to ingest files into ADLS Gen2, organised the lake using Bronze, Silver and Gold layers, and used Databricks PySpark to clean, enrich and write Delta tables. I also configured Unity Catalog external locations and solved real implementation issues such as storage permissions, ABFSS path errors and Delta table write problems. The final Gold layer is ready for SQL analysis and Power BI reporting.”

This explanation shows business value, tools used, architecture, and troubleshooting experience.

Conclusion & Next Steps

How to improve the project further

Completed outcome

- End-to-end Azure data pipeline built
- Lakehouse layers organised clearly
- Databricks transformations implemented
- Gold Delta tables ready for analysis
- Project can now be added to GitHub/portfolio

Recommended next steps

- Add screenshots of ADF, ADLS and Databricks
- Create Power BI dashboard from Gold tables
- Write a README with architecture diagram
- Add GitHub repo link in your CV
- Prepare a short demo video

Thank you